Particelle Cariche in un Campo Elettromagnetico

Gabriele Cembalo

5 gennaio 2026

Abstract

In questa esperienza ci si è occupati dello studio dell'effetto Hall. Prima di effettuare qualsiasi misura, è stato verificato che il campo magnetico generato nel traferro del magnete utilizzato fosse qualitativamente omogeneo entro una regione della sezione del magnete. Successivamente, è stata variata la corrente circolante nelle bobine per caratterizzare due cicli di isteresi completi. Dalla forma di tale ciclo è stato dedotto che il magnete fosse dolce, mentre dalle rispettive salite e discese di queste curve sono stati estrapolati i parametri della retta $B(i)$, divenuta calibrazione del set-up. In seguito, ci si è occupati del calcolo dei coefficienti di Hall di un campione di Germanio, ottenuti prima a campo magnetico costante e poi a corrente costante ed essi sono risultati compatibili. Per farlo è stato considerato anche un fondo di tensione, di natura ohmica, dovuta al disallineamento dei contatti nel semiconduttore. Per concludere, conoscendo ora il coefficiente di Hall del Germanio, è stata ricavata la mobilità dei portatori avendo prima misurato la resistenza del campione.

Indice

1 Introduzione **2**

2 Teoria **2**

3 Metodi numerici utilizzati **2**

3.1 Metodo Runge-Kutta del quart'ordine 2

3.2 Metodo Boris 3

4 Risultati nelle diverse configurazioni **3**

5 Conclusioni **3**

A Inserting Code **4**

Bibliografia **8**

1 Introduzione

In questo progetto si studia il moto di una o più particelle cariche, non relativistiche, immerse in un campo elettromagnetico costante. Vengono analizzate diverse configurazioni di campo. Nella sezione 2 analizzeremo la teoria alla base del problema. Nella sezione 3 vengono spiegate meglio le motivazioni per cui un approccio numerico è necessario/preferibile rispetto uno analitico; inoltre vengono analizzati i due metodi di risoluzione dell'equazione differenziale proposti, ovvero il metodo di Runge-Kutta del quarto ordine e l'algoritmo di Boris del secondo ordine. Studiamo le diverse configurazioni di campo elettromagnetico e i diversi risultati nella sezione 4. In particolare, considerando una particella singola, trattiamo le configurazioni: campo magnetico costante e campo elettrico nullo; campo elettrico e magnetico costanti e paralleli; campo elettrico e magnetico perpendicolari. Nell'ultimo caso proposto vengono studiate più particelle in una configurazione ad X point. Per concludere, nella sezione 5 traiamo le conclusioni. È presente nell'appendice A il codice utilizzato.

2 Teoria

3 Metodi numerici utilizzati

I metodi numerici utilizzati in questo progetto sono il metodo di Runge-Kutta del quarto ordine e l'algoritmo di Boris.

Le referenze per questi due metodi numerici sono chiaramente le slide delle lezioni, ma per il metodo Boris si è anche consultata la trattazione fatta da Qun et al. [2], consultabile al seguente link, ma una spiegazione più intuitiva, didattica e operativa può anche essere trovata in [1].

3.1 Metodo Runge-Kutta del quart'ordine

Il **metodo di Runge-Kutta del quart'ordine** viene utilizzato per la risoluzione numerica di problemi ai valori iniziali del tipo:

$$\frac{\mathrm{d}Y}{\mathrm{d}t} = R(t, Y) \qquad , \qquad Y(t_0) = Y_0. \tag{1}$$

Tale metodo è un metodo iterativo, a singolo passo, che utilizza solo l'informazione sull'intervallo corrente e non sui precedenti. Tale metodo si basa sull'espansione di Taylor, sull'approssimazione dell'integrale con la regola di Simpson ed è un buon algoritmo in termini di accuratezza e stabilità; infatti ha un'accuratezza locale dell'ordine $\mathcal{O}(\Delta t^5)$, che si traduce in un errore globale dell'ordine $\mathcal{O}(\Delta t^4)$. Il metodo richiede quattro valutazioni della *Right-Hand-Side function* per ogni passo temporale:

$$\begin{aligned}
k_1 &= R\left(t_n, Y_n\right) \\
k_2 &= R\left(t_n + \frac{\Delta t}{2}, Y_n + \frac{\Delta t}{2} k_1\right) \\
k_3 &= R\left(t_n + \frac{\Delta t}{2}, Y_n + \frac{\Delta t}{2} k_2\right) \\
k_4 &= R\left(t_n + \Delta t, Y_n + \Delta t\, k_3\right) \\
Y_{n+1} &= Y_n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).
\end{aligned} \tag{2}$$

Le quattro valutazioni della funzione R vengono effettuate in punti diversi dell'intervallo di discretizzazione temporale. In particolare, k_1 e k_4 corrispondono rispettivamente agli estremi iniziale e finale dell'intervallo, mentre k_2 e k_3 sono calcolati nel punto medio, fornendo una stima più accurata dell'andamento della soluzione.

La combinazione pesata dei coefficienti k_i deriva dall'approssimazione dell'integrale mediante la regola di Simpson e consente di ottenere un'elevata accuratezza senza aumentare il numero di passi temporali.

3.2 Metodo Boris

Il secondo metodo utilizzato è il **metodo di Boris**, che è un integratore simplettico e un'estensione dell'algoritmo Position-Verlet. Questo metodo viene utilizzato per la risoluzione numerica del problema ai valori iniziali associato alle equazioni del moto di una particella carica soggetta a campi elettromagnetici, ovvero a problemi in cui la forza dipende dalla velocità, come nel caso descritto dal sistema:

$$\begin{cases} \dfrac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{v} \\ \dfrac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = \dfrac{q}{m}\Big(\mathbf{E} + \mathbf{v} \times \mathbf{B}\Big) \end{cases} \quad , \qquad \mathbf{x}(t_0) = \mathbf{x}_0, \quad \mathbf{v}(t_0) = \mathbf{v}_0 \tag{3}$$

in cui $\mathbf{E}$ e $\mathbf{B}$ indicano rispettivamente il campo elettrico e magnetico valutati nella posizione della particella. Il metodo di Boris è un metodo iterativo, esplicito e a singolo passo, che utilizza esclusivamente le informazioni relative all'intervallo temporale corrente e non sui passi precedenti. A differenza dei metodi di tipo Runge-Kutta, esso non si basa su un'espansione di Taylor né su una quadratura diretta dell'integrale, ma su una scomposizione simmetrica dell'operatore di avanzamento temporale, che separa l'azione del campo elettrico da quella del campo magnetico.

Il metodo è di secondo ordine in accuratezza globale, con errore locale dell'ordine $\mathcal{O}(\Delta t^3)$, ed è particolarmente apprezzato per le sue proprietà di stabilità e per la capacità di preservare le invarianti fisiche del sistema. In particolare, in assenza di campo elettrico, il metodo conserva l'energia cinetica della particella, riproducendo correttamente il moto di rotazione indotto dal campo magnetico.

L'aggiornamento della velocità viene effettuato attraverso una sequenza di tre sotto-passi, mentre la posizione viene aggiornata come:

$$\begin{aligned} \mathbf{x}_{n+\frac{1}{2}} &= \mathbf{x}_n + \frac{\Delta t}{2}\mathbf{v}_n \\ \mathbf{v}_- &= \mathbf{v}_n + \frac{\Delta t}{2}\mathbf{E}_{n+\frac{1}{2}} \\ \mathbf{v}_+ &= \mathbf{v}_- + 2\frac{\mathbf{v}_- + \mathbf{v}_- \times \frac{\Delta t}{2}\mathbf{B}_n}{1 + \frac{\Delta t^2}{4}\mathbf{B}_n^2} \times \left(\frac{\Delta t}{2}\mathbf{B}_n\right) \\ \mathbf{v}_{n+1} &= \mathbf{v}_+ + \frac{\Delta t}{2}\mathbf{E}_{n+\frac{1}{2}} \\ \mathbf{x}_{n+1} &= \mathbf{x}_{n+\frac{1}{2}} + \frac{\Delta t}{2}\mathbf{v}_{n+1}. \end{aligned} \tag{4}$$

Gli aggiornamenti intermedi della velocità vengono effettuati in punti diversi dell'intervallo di discretizzazione temporale. In particolare, i termini $\mathbf{v}_-$ e $\mathbf{v}_{n+1}$ rappresentano gli effetti del campo elettrico applicati rispettivamente all'inizio e alla fine del passo temporale, mentre l'aggiornamento centrale $\mathbf{v}_+$ descrive l'azione del campo magnetico, trattata come una rotazione nel punto medio dell'intervallo.

La struttura simmetrica del metodo consente una buona conservazione delle invarianti fisiche del sistema, in particolare dell'energia cinetica in assenza di campo elettrico, garantendo stabilità numerica anche per integrazioni di lungo termine.

N.B. Nell'implementazione dell'algoritmo nell'Appendice A viene utilizzata la notazione (e le quantità) di [1]. Ci serviremo delle quantità:

$$t = \frac{\Delta t}{2}\mathbf{B} \quad , \quad \mathbf{v}' = \mathbf{v}_- + \mathbf{v}_- \times \mathbf{t} \quad , \quad \mathbf{s} = \frac{2\mathbf{t}}{1 + t^2} \tag{5}$$

così viene comodo calcolare:

$$\mathbf{v}_+ = \mathbf{v}_- + \mathbf{v}' \times \mathbf{s}. \tag{6}$$

4 Risultati nelle diverse configurazioni

5 Conclusioni

A Inserting Code

```
////////////////////////////////////////////////////////////////////////////////
//              Name    : Gabriele Cembalo                                    //
//              A.Y.    : 2025-2026                                           //
//              Project : Particle(s) in Electromagnetic Fields               //
//                                                                            //
//              Starting date : 24-12-2025                                    //
//              Ending date   : ..-..-2026                                    //
////////////////////////////////////////////////////////////////////////////////

#include <iostream>
#include <iomanip>
#include <cmath>
#include <fstream>

using namespace std;

////////////////////////////////////////////////////////////////////////////////
//                            Macro declaration                               //
////////////////////////////////////////////////////////////////////////////////
# define STAGE 4 // Choose the problem type (1 to 4)
                 // 1,2,3 : single particle
                 // 4 : multiple particles (N_Part)
#define NMAX_EQ 64 // Maximum number of elements (safety limit)
                   // to avoid variable-size arrays
#define N_Part 1.e3 // Number of particles in the multi-particle problem
#define X_L 1000 // Length of the X axis for STAGE 4
#define Y_L 1000 // Length of the Y axis for STAGE 4
#define TL 100.0 // Simulation's final time

////////////////////////////////////////////////////////////////////////////////
//                          Functions declaration                             //
////////////////////////////////////////////////////////////////////////////////
void EB_Fields(double *, double *);
void InitialCondition(double *);
void RHSFunc(double, double *, double *, double *);
void RK4Step(double, double *, double *,
             void (*)(double, double *, double *, double *),
             double, int);
void BorisStep(double, double *, double *, double);

////////////////////////////////////////////////////////////////////////////////
//                        Main function implementation                        //
////////////////////////////////////////////////////////////////////////////////
int main(){

    ofstream fdata;
    fdata.open("file.dat"); // Solutions file

    fdata << setiosflags ( ios::scientific );
    fdata << setprecision ( 10 );

    cout << setiosflags ( ios::scientific );
    cout << setprecision ( 7 );

    int neq = 6; // Number of equations

    double Y[neq]; // (x, y, z)    = (Y[0], Y[1], Y[2])
                   // (vx, vy, vz) = (Y[3], Y[4], Y[5])

    double EB[neq]; // Electromagnetic field
                    // (Ex, Ey, Ez) = (EB[0], EB[1], EB[2])
                    // (Bx, By, Bz) = (EB[3], EB[4], EB[5])

    return 0;

}

```

```
////////////////////////////////////////////////////////////////////////////////
//                          Functions implementation                          //
////////////////////////////////////////////////////////////////////////////////


////////////////////////////////////////////////////////////////////////////////
// Electric and magnetic fields.                                              //
// Input: pointer to the solution vector,                                     //
//        pointer to the electric and magnetic field vector.                 //
////////////////////////////////////////////////////////////////////////////////
void EB_Fields(double *Y, double *EB){

    double l = 0.001; // Value of 1/L, where L = X_L = Y_L

    #if STAGE == 1 // Single particle problem
                   // Simple gyration
        EB[0] = 0.0 ; EB[1] = 0.0 ; EB[2] = 0.0;
        EB[3] = 0.0 ; EB[4] = 0.0 ; EB[5] = 1.0;

    #elif STAGE == 2 // Single particle problem
                     // ExB drift in parallel configuration ( |E| < |B| )
        EB[0] = 0.0 ; EB[1] = 0.0 ; EB[2] = 0.8;
        EB[3] = 0.0 ; EB[4] = 0.0 ; EB[5] = 1.0;

    #elif STAGE == 3 // Single particle problem
                     // ExB drift in perpendicular configuration ( |E| < |B| )
        EB[0] = 0.0 ; EB[1] = 0.8 ; EB[2] = 0.0;
        EB[3] = 0.0 ; EB[4] = 0.0 ; EB[5] = 1.0;

    #elif STAGE == 4 // Multiple particles problem
                     // X point
        EB[0] = 0.0 ; EB[1] = 0.0 ; EB[2] = 0.5;
        EB[3] = Y[1]*l ; EB[4] = Y[0]*l ; EB[5] = 0.0;

    #endif

}


////////////////////////////////////////////////////////////////////////////////
// Initial condition.                                                          //
// Input: pointer to the solution vector.                                      //
////////////////////////////////////////////////////////////////////////////////
void InitialCondition(double *Y){

    #if STAGE == 1 || STAGE == 2 || STAGE == 3 // Single particle problem
        Y[0] = 1.0 ; Y[1] = 0.0 ; Y[2] = 0.0; // Initial position
        Y[3] = 0.0 ; Y[4] = 1.0 ; Y[5] = 0.0; // Initial velocity

    #elif STAGE == 4 // Multiple particle problem
        // Setting random initial conditions
        // with a constraint on the velocity magnitude.
        double x, y, v, vx, vy, theta;
        v = 0.1;

        // Generating random (x, y) coordinates in the range [-1000, 1000]^2
        x = (drand48()*2.0*X_L) - X_L;
        y = (drand48()*2.0*Y_L) - Y_L;

        theta = drand48()*2*M_PI; // Random angle in [0,2*pi]

        // Random velocity with magnitude v and angle theta
        vx = v*cos(theta);
        vy = v*sin(theta);

        // Filling the state vector
        Y[0] = x ; Y[1] = y ; Y[2] = 0.0;   // Initial position
        Y[3] = vx ; Y[4] = vy ; Y[5] = 0.0; // Initial velocity

```

```
    #endif

}


////////////////////////////////////////////////////////////////////////////
// Right-Hand-Side function of the equation.                              //
// Input: integration variable,                                           //
//        pointer to the solution vector,                                 //
//        pointer to the electric and magnetic field vector,              //
//        pointer to the RHS vector.                                      //
////////////////////////////////////////////////////////////////////////////
void RHSFunc(double t, double *Y, double *EB, double *R){

    // Using natural units: c = 1

    EB_Fields(Y, EB); // Fill the EB vector

    // Assigning variables
    double x = Y[0] , y = Y[1], z = Y[2];
    double vx = Y[3] , vy = Y[4] , vz = Y[5];
    double Ex = EB[0] , Ey = EB[1] , Ez = EB[2];
    double Bx = EB[3] , By = EB[4] , Bz = EB[5];

    R[0] = vx;
    R[1] = vy;
    R[2] = vz;
    R[3] = Ex + vy*Bz - vz*By;
    R[4] = Ey + vz*Bx - vx*Bz;
    R[5] = Ez + vx*By - vy*Bx;

}


////////////////////////////////////////////////////////////////////////////
// Fourth-order Runge-Kutta method.                                       //
// Input: integration variable,                                           //
//        pointer to the solution vector,                                 //
//        pointer to the electric and magnetic field vector,              //
//        pointer to the Right-Hand-Side-Function,                        //
//        step size,                                                      //
//        order of the ODE system.                                        //
////////////////////////////////////////////////////////////////////////////
void RK4Step(double t, double *Y, double *EB,
             void (*RHSFunc)(double t, double *Y, double *EB, double *R),
             double h, int neq){

    int i; // Loop index
    // Vectors for intermediate steps
    double Y1[NMAX_EQ], k1[NMAX_EQ], k2[NMAX_EQ], k3[NMAX_EQ], k4[NMAX_EQ];

    RHSFunc(t, Y, EB, k1); // Computation of k1 using RHS at t_n and Y_n

    // Loop to compute Y_n + k1*h/2
    for(i = 0 ; i < neq ; i++ ){

        Y1[i] = Y[i] + 0.5*h*k1[i];

    }

    RHSFunc(t+0.5*h, Y1, EB, k2); // Computation of k2 using RHS at t_n + h/2
                                  // and Y_n + k1*h/2

    // Loop to compute Y_{n+1} = Y_n + k2*h/2
    for(i = 0 ; i < neq ; i++){

        Y1[i] = Y[i] + h*k2[i]*0.5;

    }
```

```

    RHSFunc(t+0.5*h, Y1, EB, k3); // Computation of k3 using RHS at t_n + h/2
                                  // and Y_n + k2*h/2

    // Loop to compute Y_{n+1} = Y_n + k3*h
    for(i = 0 ; i < neq ; i++){

        Y1[i] = Y[i] + h*k3[i];

    }

    RHSFunc(t+h, Y1, EB, k4); // Computation of k4 using RHS at t_n + h
                              // and Y_n + k3*h

    // Loop to compute Y_{n+1} = Y_n + h/6 * ( k1 + 2*k2 + 2*k3 + k4 )
    for(i = 0 ; i < neq ; i++){

        Y[i] += h * ( k1[i] + 2.0*k2[i] + 2.0*k3[i] + k4[i] ) / 6.0;

    }

}


////////////////////////////////////////////////////////////////////////////////
// Boris Method Step (second-order, symplectic)                               //
// Input: integration variable,                                               //
//        pointer to the solution vector,                                     //
//        pointer to the electric and magnetic field vector,                  //
//        step size.                                                          //
////////////////////////////////////////////////////////////////////////////////
void BorisStep(double t, double *Y, double *EB, double h){

    // Reminder: (x, y, z)    = (Y[0], Y[1], Y[2])
    //           (vx, vy, vz) = (Y[3], Y[4], Y[5])
    //           (Ex, Ey, Ez) = (EB[0], EB[1], EB[2])
    //           (Bx, By, Bz) = (EB[3], EB[4], EB[5])

    // We'll compute the following step: 1. x_{n+1/2} = x_n + v_n * h/2
    //                                   2. s and t
    //                                   3. v_minus = v_n + E * h/2
    //                                   4. v_prime and v_plus
    //                                   5. v_{n+1} = v_plus + E * h/2
    //                                   6. x_{n+1} = x_{n+1/2} + v_{n+1} * h/2

    int i; // Loop index
    // Vectors for intermediate steps
    double v_minus[3], v_plus[3], t_vec[3], s_vec[3], v_prime[3];
    // Where: t_vec = B * h * 0.5
    //        s_vec = 2t / ( 1 + |t|^2 )
    //        v^prime = v_- + v_- cross t
    double t_mag; // This is the magnitude ^2 of t_vec

    // 1. Compute the position half-step: x_{n+1/2} = x_n + v_n * h/2
    for( i = 0 ; i < 3 ; i++ ){

        Y[i] = Y[i] + Y[i+3] * 0.5 * h;

    }

    // 2. Compute the intermediate vectors for the magnetic rotation
    t_mag = 0;
    for( i = 0 ; i < 3 ; i++ ){

        t_vec[i] = EB[i+3] * 0.5 * h;
        t_mag += t_vec[i] * t_vec[i];

    }
    for( i = 0 ; i < 3 ; i++ ){
```

```
        s_vec[i] = 2.0 * t_vec[i] / (1.0 + t_mag);

    }

    // 3. Compute the velocity v_- step (electric field):
    //     v_minus = v_n + E * h/2
    for( i = 0 ; i < 3 ; i++ ){

        v_minus[i] = Y[i+3] + EB[i] * 0.5 * h;

    }

    // 4. Magnetic rotation (Boris Trick)
    //     v_prime = v_minus + v_minus cross t
    v_prime[0] = v_minus[0] + (v_minus[1] * t_vec[2] - v_minus[2] * t_vec[1]);
    v_prime[1] = v_minus[1] + (v_minus[2] * t_vec[0] - v_minus[0] * t_vec[2]);
    v_prime[2] = v_minus[2] + (v_minus[0] * t_vec[1] - v_minus[1] * t_vec[0]);

    // Compute: v_plus = v_minus + v_prime x s
    v_plus[0] = v_minus[0] + (v_prime[1] * s_vec[2] - v_prime[2] * s_vec[1]);
    v_plus[1] = v_minus[1] + (v_prime[2] * s_vec[0] - v_prime[0] * s_vec[2]);
    v_plus[2] = v_minus[2] + (v_prime[0] * s_vec[1] - v_prime[1] * s_vec[0]);

    // 5. Compute the second velocity step (electric field):
    //     v_{n+1} = v_plus + E * h/2
    for( i = 0 ; i < 3 ; i++ ){

        Y[i+3] = v_plus[i] + EB[i] * 0.5 * h;

    }

    // 6. Compute the position step: x_{n+1} = x_{n+1/2} + v_{n+1} * h/2
    for( i = 0 ; i < 3 ; i++ ){

        Y[i] = Y[i] + Y[i+3] * 0.5 * h;

    }

}
```

Riferimenti bibliografici

[1] Particle In Cell Consulting. *The Boris Method: $\boldsymbol{v} \times \boldsymbol{B}$ Rotation*. https://www.particleincell.com/2011/vxb-rotation/. 2011.

[2] Hong Qin et al. «Why is Boris algorithm so good?» In: *Physics of Plasmas* 20.8 (2013), p. 084503. DOI: 10.1063/1.4818428.